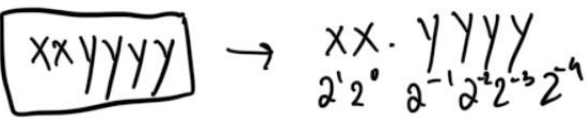


• Quantization for CS61C → Where do floating points come into picture

CS61C Review:

- So far we have gone through binary points & floating binary points, as a quick review

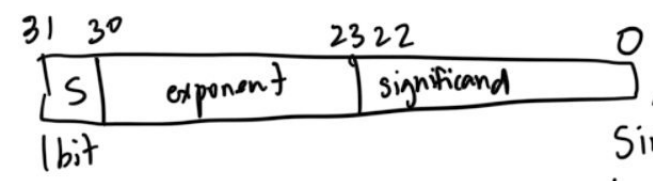
A simple 6-bit representation in binary point for positive numbers is



Enter floating points, which is a computer representation of real numbers using binary scientific notation, and in programming languages like C, Java, C++, & Python, floating point representation is used for float and double data types

In CS61C we provide students how to represent a float or double in 32 bits, using the IEEE 754 Single-Precision Floating Point Standard:

- 1st bit is the sign
- Next 8 bits are the exponent



Single precision standard for a 32-bit word

- The last 23 bits are for the significand
- To decimal you follow:
- To have equal range of negative and positive values represented
- (exponent - 127)

$$(-1)^S \times (1 + \text{significand}) \times 2^{(\text{exponent} - 127)}$$

That's hopefully a quick review for those who have taken 61C or are currently taking it and would need a quick refresh!! But why do all these converting or unconvverting from this decimal to binary stuff?

If you've taken a 61C exam, you have probably converted a decimal number into IEEE 754 by hand ~~and~~ determine the sign $\rightarrow$ normalize the binary value $\rightarrow$ calculate the biased exponent $\rightarrow$ fill in the fraction bits

It often feels like an isolated mechanical exercise that with more reps you can ace in an exam, instead of an actual decision that affects real software out there

BUT, the format used to represent a number changes what a computer can do with it.

Lets say we're given a fixed number of bits. You must decide how to exactly divide those bits b/w representing very LARGE and very small values, distinguishing b/w very nearby values and also recording the sign; overall there are 4 categories to consider:

- | | | | |
|---|---|--|--|
| 1. Range
↓
largest & smallest magnitudes we can represent | 2. Precision
↑
How closely we can distinguish nearby values | 3. Storage
↓
How many bytes every value occupies | 4. Execution
- Hardware instructions that can process the value efficiently |
|---|---|--|--|

* These choices have become very visible in machine learning. Tons of models out there can contain up to billions of numerical parameters. Now if we follow the standard taught in 61C, then each numerical parameter occupies 4 bytes (32bits)

Here's some math: If we have a 1 Billion parameter model, then it takes up

$$\underbrace{10^9 \text{ parameters}}_{\text{[weights \& biases combined]}} \times 4 \text{ bytes per weight} \approx 4 \text{ GB}$$

What if each parameter occupied 2 bytes? In other words, what would happen if each numerical parameter was represented in 16 bits instead of 32? Then that would approximately half the storage. And then what about one byte (8 bits)? It reduces the size again by $1/2$ to 1 GB space. These different representations change how much data overall must travel through memory and whether the processor has instructions designed for that datatype (32 bits, 16 bits, 8 bits, etc.)

This is why machine learning uses formats such as FP32, FP16, BF16, FP8, INT8. But what do all these names actually mean and why do we need so many formats? Are they simply just cutting off a few decimal places? Not exactly.

Each of these formats make a different compromise in range, memory, precision, and hardware execution. Here's a diagram of representing one of my favorite numbers (well its in decimal in this case) in 3 ways:

Mathematical Value we want to store: 0.93

Diagram 1

0.93

FP32

0 | 01111101 | 101110000101000111011
s | exponent | fraction

Stored Value: 0.9300000072

Storage: 4 Bytes

FP16

0 | 01110 | 1101110001
s | exp. | fraction

Stored Value: ~0.9301757813

Storage: 2 bytes

INT8

8 bits
+ shared scale

$$s = \frac{1}{127} \quad q = \text{round}\left(\frac{0.93}{s}\right) = 118 \\ \hat{x} = 118 \left(\frac{1}{127}\right) \\ \approx 0.92913$$

FP32 & FP16 encode their scale inside each value using an exponent

INT8 stores an integer and relies on separate quantization metadata to recover an approximate real value

Range vs. Precision

Here's a quick review of floating point formats from CS61C: Floating point formats are able to represent extremely large values & small differences between these nearby values by dividing their available bits into three fields:

$$(-1)^s \times 1.f \times 2^{e-\text{bias}}$$

For ordinary normalized values:

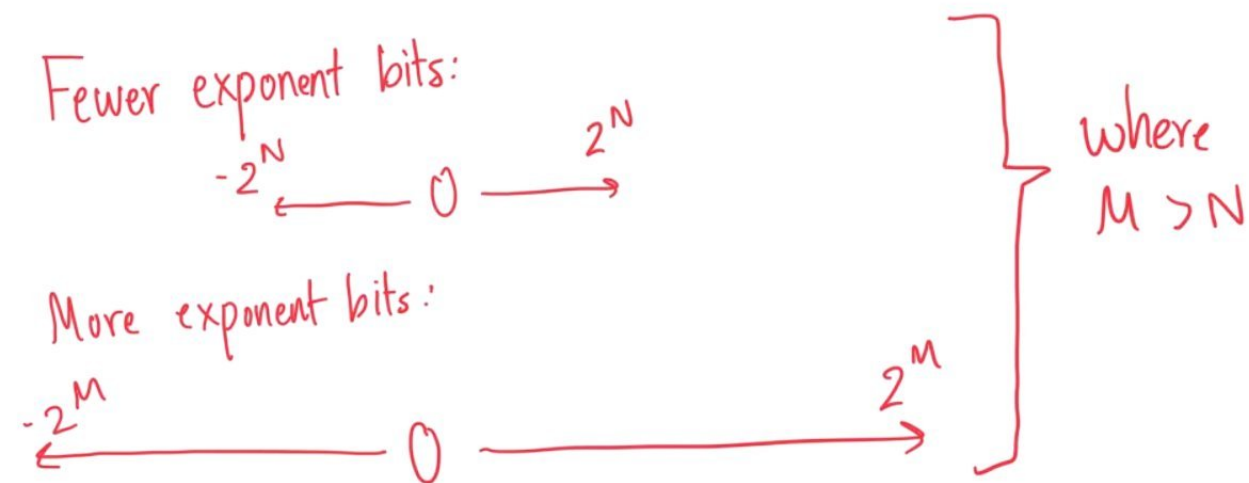
- $s \rightarrow$ determines sign
- $e \rightarrow$ determines scale of the number
- $f \rightarrow$ precision within that scale

A useful interpretation I tell students is that the exponent chooses the general "neighborhood" on this number line, while the fraction f chooses the specific position within that neighborhood

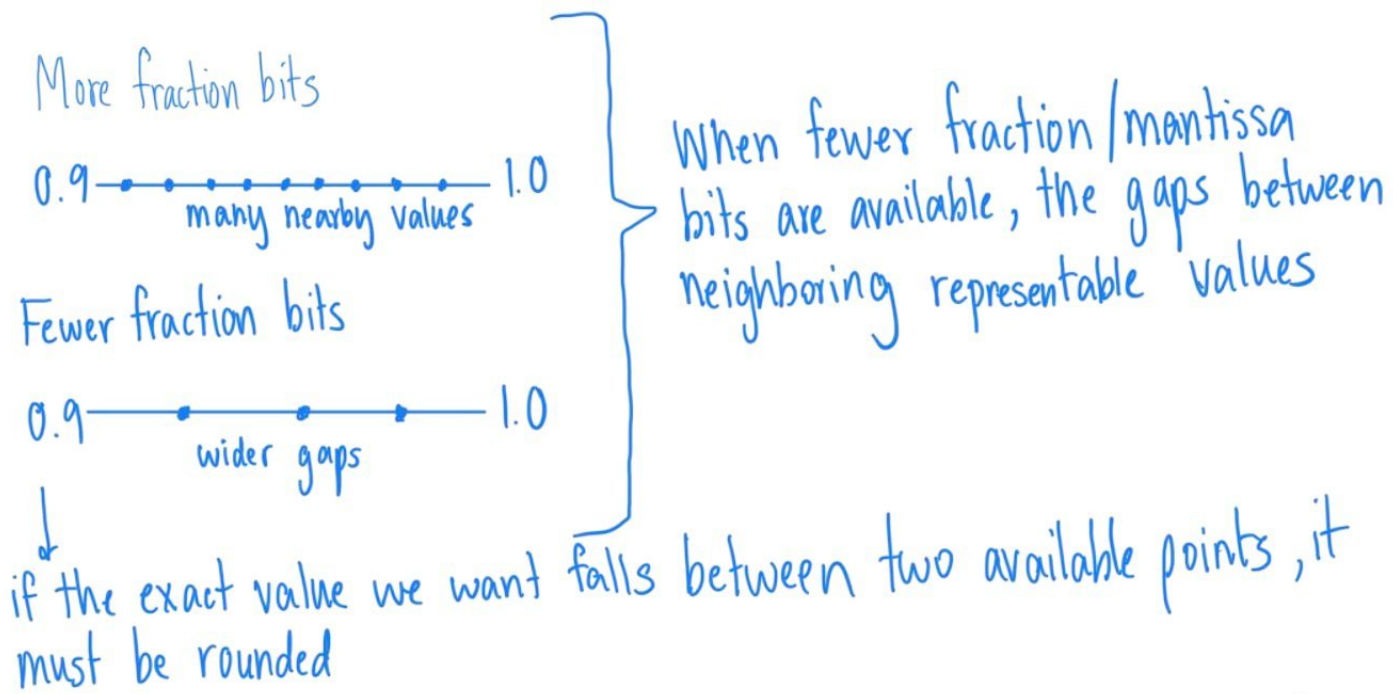
There are tradeoffs with deciding between more bits for exponent or more bits for the fraction

1. Exponent bits provide range

Having more bits in the exponent section represents a much wider range in magnitudes. This is why BF16 has approximately the same numerical range as FP32: both utilize 8 bits for the exponent



2. Fraction bits provide precision
On the other hand, having more fraction section bits allows more values to be represented for a given interval.



The FP16 format stores 10 fraction bits, while BF16 stores seven. So BF16 instead spends more of its limited bit budget on exponent range

Quick Note: The term BF, or format BF16, is short for bfloat16, or "Brain Floating Point". It is a 16-bit floating point format originally developed by Google Brain specifically for machine-learning workloads. It is NOT the same format as IEEE FP16, to compare:

FP16: more fraction bits, narrower range

BF16: fewer fraction bits, wider range

3. Total Bits Determine Storage

The total width (bits) determines how much memory each stored value occupies:

Table Using Sheets

8-bit floating point formats (FP8) further push the same tradeoff as FP16 and BF16. Ex, FP8 E4M3 (M denotes to mantissa bits, still corresponds to the stored fraction field) has more bits for precision than E3M2. I'll leave FP8 for now, so I can further explain the basics of quantization from here!

INT8 representation is different. In INT8, the distance between any two numbers is always locked at 1.0 $(-2, -1, 0, 1, 2, \dots)$, the value can only store an integer usually between -128 and 127.

However, neural network weights often look like 0.037, -0.62, or of course 0.93 lol. So if hardware only supports INT8, How can an integer format store them?

We can't just convert small weights to integers, that would just destroy most of the information: $0.93 \rightarrow 1$, $0.26 \rightarrow 0$, $-0.26 \rightarrow 0$, many different values will just collapse to a small integer

The key idea here is an integer does not have to mean "one whole unit". If you have a scale s , then one integer step can instead represent a smaller real-valued interval

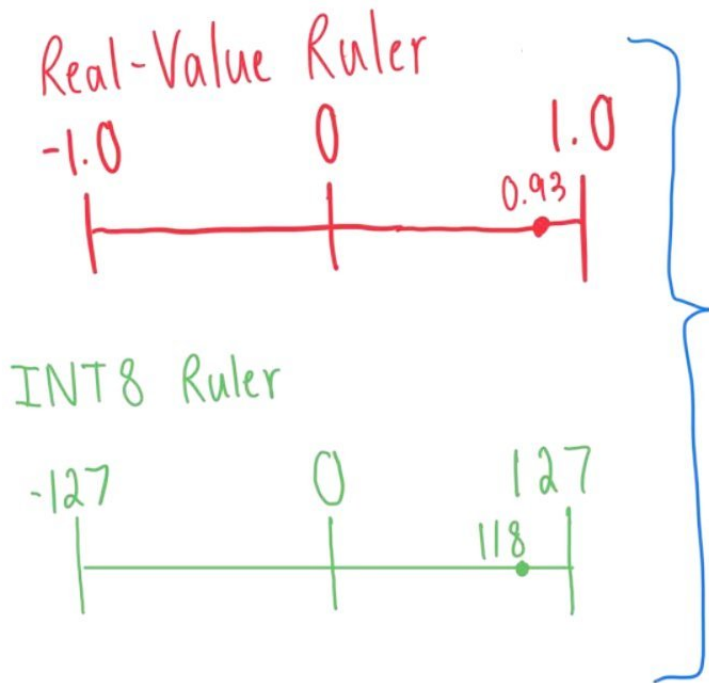
Since INT8 ranges from -128 to 127, we'll set the scale s to

$$s = \frac{1}{127} \approx 0.00787$$

Now we can interpret integer 1 as ~ 0.00787 , 10 as 0.0787 , 127 as 1.0

To store 0.93, the q representation for INT8 would be $\text{round}\left(\frac{0.93}{1/127}\right) = 118$ and to reconstruct we do $\hat{x} = 118 \left(\frac{1}{127}\right) \approx 0.929$

INT8 Diagram



Now every weight no longer needs its own exponent and fraction fields. The only cost is approximation error

Overall, floating point gives each value its own exponent, allowing values of very different magnitudes to have its own form of representation. INT8 gives a group of integers a shared scale. This makes the representation smaller, but forces many original values onto a limited set of intervals. This entire mapping process is also known as

QUANTIZATION

Section 3: Quantization

Finally we are here talking about the fun 'juicy' stuff: Quantization!!
So, what exactly is quantization? Well, based on the

previous section:

Quantization maps values from a larger set of possible numerical values onto a smaller set of representable values

This can happen in many ways:

- FP32 value can be rounded to a value representable in FP16, BF16, or FP8
- A floating point can also be represented by an integer and a shared scale
- Weights can be restricted to an EVEN smaller set, such as binary values $\{-1, +1\}$ or ternary values $\{-1, 0, +1\}$

Floating-point Reduction: FP32 to FP16

Consider 0.93, its FP32 representation is

0 | 01111110 | 11011100001010001111011

S | exponent | fraction

This stored FP32 exponent is 126

Subtract bias of 127, unbiased exponent

is now $126 - 127 = -1$

FP16 must now represent the same exponent using its bias of 15: $-1 + 15 = 14$

14 in five bits is 01110

FP16 can only store 10 bits instead of 23, so the fraction is shortened & rounded to

FP16: 1101110001

Resulting FP16 representation is:

0 | 01110 | 1101110001

→ This bit pattern now represents 0.93017578125

This conversion maps FP32 approximation of 0.93 to the nearest available FP16 value.

As shown in the previous section, integer quantization works differently.

Given an original weight w and scale s , we can calculate the integer to store as

$$q = \text{round}\left(\frac{w}{s}\right)$$

where w is the original value, s is the scale, and q is the stored integer

And to approximately reconstruct the original value, we calculate

$$\hat{w} = sq$$

Reconstruction opens up one issue, which is quantization error, the difference between an original value and its reconstructed approximation, where

$$\epsilon = w - \hat{w}, \text{ and size of error is } |\epsilon| = |w - \hat{w}|$$

Quantization loses information because multiple original values may map to the same stored representation. It highlights the affect the scale value has on the quantization error, as the scale determines which range of values these patterns represent.

Let's consider a linear layer with weight matrix:

$$W \in \mathbb{R}^{\text{dout} \times \text{din}}$$

where each row corresponds to one output feature

	<u>input features</u>
output	$[w_{11} \ w_{12} \ w_{13} \ w_{1n}]$
features	$[w_{21} \ w_{22} \ w_{23} \ w_{2n}]$
↓	$[w_{31} \ w_{32} \ w_{33} \ w_{3n}]$

A scale can be applied at different levels of scale granularity:

• Per-tensor scaling:

One scale is shared across the entire weight tensor

$$[w_{11} w_{12} w_{13} \dots w_{23} w_{23}] \times \text{scale}_1 \quad \left. \vphantom{[w_{11} w_{12} w_{13} \dots w_{23} w_{23}]} \right\} \begin{array}{l} \text{The scale must} \\ \text{accommodate the tensor's} \\ \text{complete value range} \end{array}$$

• Per-channel scaling: Each channel receives a separate scale:

$$[w_{11} w_{12} w_{13}] \times \text{scale}_1$$

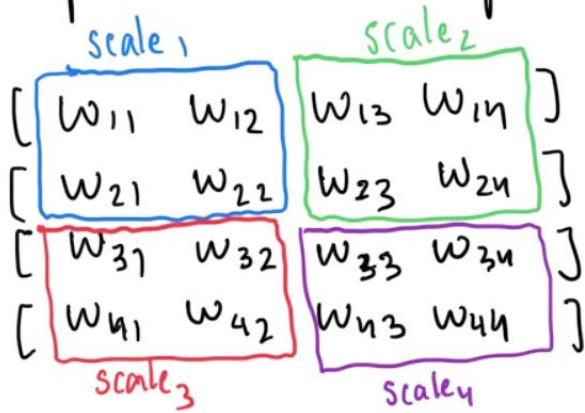
$$[w_{21} w_{22} w_{23}] \times \text{scale}_2$$

This scaling can either be per-input-channel (column) or per output channel (row). Most modern AI accelerators strongly prefer per-output-channel quantization for model weights

Group-Wise: divides each channel into smaller groups and assigns a scale to each group (1D)

row 1 $\left[\overset{\text{scale}_1}{\boxed{w_{11} \ w_{12}}} \ \overset{\text{scale}_2}{\boxed{w_{13} \ w_{14}}} \right] \rightarrow \text{smaller one dimensional groups}$

Tile-Wise Scaling: Partitions the matrix in two dimensions;
one scale spans multiple rows and columns
Example for a tile shape of 2×2 :



Smaller groups more closely match the local ranges, which can reduce error. However, having more scales is additional scaling metadata and computation

Larger groups $\rightarrow$ Less metadata, potentially more error

Smaller groups $\rightarrow$ More metadata, potentially less error

What parts of a model can be quantized?

During inference, three major categories are commonly quantized:

1. **Weights**: as shown as an example above, weights are learned parameters stored with the model

2. **Activations**: Temporary values produced while processing the current input, example is the tensor coming out of the ReLU activation function

3. **KV cache**: keys and values saved from previous tokens, technically it's info of stored intermediate activations, but takes up a lot of space in memory as batch size grows